Property-Based Testing

A little recursive function

```
let rec take (n : int) (xs : int list) : int list =
  if n <= 0 then []
  else begin match xs with
             | [] -> []
             | x :: xs' -> x :: take (n - 1) xs'
       end
```

```
take 2 [1;2;3;4]  =  [1;2]
take 0 [1;2;3;4]  =  []
take 6 [1;2;3;4]  =  [1;2;3;4]
```

Some unit tests

```
let test () : bool =
  take 0 [1;2;3] = []
;; run_test "..." test

let test () : bool =
  take 0 [] = []
;; run_test "..." test
```

```
let rec take (n : int) (xs : int list) : int list =
  if n <= 0 then []
  else begin match xs with
             | [] -> []
             | x :: xs' -> x :: take (n - 1) xs'
       end
```

A common pattern

```
let test () : bool =
  take 0 [1;2;3] = []
;; run_test "..." test

let test () : bool =
  take 0 [] = []
;; run_test "..." test
```

Seems like applying `take 0` to *any* list yields an empty list…

```
let rec take (n : int) (xs : int list) : int list =
  if n <= 0 then []
  else begin match xs with
             | [] -> []
             | x :: xs' -> x :: take (n - 1) xs'
       end
```

Generalize the pattern

```
let property (xs : int list) : bool =
  take 0 xs = []

let test () : bool = property [1;2;3]
;; run_test "..." test

let test () : bool = property []
;; run_test "..." test
```

Define a generic **property**

Instantiate it for multiple tests

```
let rec take (n : int) (xs : int list) : int list =
  if n <= 0 then []
  else begin match xs with
            | [] -> []
            | x :: xs' -> x :: take (n - 1) xs'
       end
```

Why was this step useful?

1. We've extracted the common pattern from multiple unit tests into a general **property** of the function.
   1. Helps us think about / document what the function **does**
   2. More concise than previous unit tests

      → Makes it easy to write *more* unit tests

2. Since each test case now just consists of single list, we can generate a lot of test cases **randomly**!

Generators

```
;; print_samples
     gen_int
     5
     string_of_int
```

A **generator** for random ints

```
Generated: 36
Generated: 22
Generated: 3
Generated: 38
Generated: 98
```

Generators

A generator for random **lists of ints**

```
;; print_samples
     (gen_list gen_int)
     5
     (string_of_list string_of_int)
```

```
  Generated: [6; -9; 8; 8]
  Generated: [-7; -6; -7; 25; -1; -3; 8; 9; 8; 35; 3; 4; -88; 94; 71; 9; -7; 5; 20; -4; -9; 15; -4; 0; 78; -4; -16; -29; 9; -18; 0; 9; 6;
16; 1; -7; -3; 83; -41; 2; 94; 0; -5; 5; 0; 1; -4; 7; 8; 7; -7; -1; -9; -8; -6; 6; -49; -5; 4; -28; -9; 7; 0; 0; -2; 2; 7; -7; 1; 0; 9; -7;
-5; 9; -33; -9; 0; -3; 9; -6; 54; -3; 44; 2; -1; -8; 61; -13; -9; -1; 3; -5; 81; 5; -5; -4; 2; 8; 7; -5; -3; 5; 3; 4; 98; 2; -9; 72; 8; -8;
-7; -8; -64; 4; -1; 9; -4; -58; -5; 88; 2; 1; -5; -4; -9; -5; 4; 8; -3; 0; 2; -77; 0; 8; 0; -7; 4; 92; -47; -6; 1; 0; -74; -5; 2; -3; -1; -
7; 4; -76; 0; -97; 9; -5; -3; -1; 2; -5; 1; -22; 6; -1; 8; 1; -2; -4; 6; -2; 0; 3; 0; 20; -2; 1; 4; -38; 5; 9; 40; -6; 6; -1; -25; -8; 6;
9; -8; -5; 1; -7; 9; 4; 8; 9; -1; 0; 4; 3; 3; 19; -4; 0; 4; 0; -4; 2; 8; -2; -98; 0; -9; -8; -3; -2; 2; 7; -72; -2; -2; 3; 0; 0; 8; -8; 0;
0; -3; 3; -5; -6; -5; -3; -6; -4; -2; -90; -9; 69; -36; -35; -8; 7; 0; -5; 6; -8; -7; -4; -5; -3; -8; -57; -2; -4; 5; 7; -7; 4; 65; -66; 2;
-2; 69; 3; 57; 2; 1; -3; 9; 3; 5; 4; 69; 0; 3; -2; 1; -3; 9; -51; 66; 8; -8; 2; -4; -67; -6; -73; 6; -6; 7; -4; -4; 41; 1; -6; 44; -6; 6;
8; -7; -8; 0; -1; -1; 4; -60; -5; 3; 4; -9; 5; -99; -2; -73; -5; 50; 0; -4; -7; -9; -54; -9; 2; 1; 7; 92; -5; 8; 6; -7; 6; -40; 6; 75; 0; -
1; 19; -8; 0; 0; -6; 1; -33; 0; 2; 2; -7; -4; 22; 4; -1; 0; 48; 86; 0; 7; -51; 5; -6; 44; -2; -6; -25; 2; -4; 2; -28; 86; -8; 8; 7; -8; 2;
5; 2; -7; -2; -3; -52; 0; -6; 3; -6; 94; 4; 5; 8; 3; -82; 4; 2; -3; 9; -8; 9; -3; -1; -7; -8; -6; -12; -6; -1; -3; 9; -7; 4; -1; -6; 93; 7;
-9; -4; -1; -2; 0; -11; -6; -9; -1; -3; -1; -56; 12; -7; -7; -3; 3; 6; 4; -8; 9; 2; -5; 9; 1; 5; -6; 77; -30; 8; 31; 6; -82; 9; 63; 6; -8;
5; 7; 74; -64; 62; 8; -4; -8; 8; -60; 0; 98; 84; 0; 7; -1; 84; -2; -2; -2; 3; -8; 0; 92; 29; 6; -4; 8; -9; -23; -2; 7; -9; -4; 9; 8; 10; 0;
-95; 8; -5; 9; 4; -8; -2; -9; 8; 94; 0; 4; 0; -3; -9; 5; 7; 0; 21; 1; 83; 6; -3; -6; -3; 3; -14; 3; 6; 11; 9; 2; -53; 72; -55; -9; 0; 0; -
5; 31; -5; 6; -7; 7; 34; 53; -2; 1; -5; -5; 78; -6; -4; -70; 0; 48; 7; -7; 4; -55; -7; 0; 0; 2; -4; 0; 5; -6; 7; -2; -2; -1; 9; 7; 3; 9;
94; 7; 58; -38; 84; 9; 31; 1; -2; 30; 5; 28; 0; 28]
  Generated: [9; -8; 4; 4; 0]
  Generated: [7; -8; -5; -3; -3; 0; -5; 8; 48; 9; 84; 7; 3; 0; -6; 43; -7; -84; -7; -6; -5; 6; -7; 8; 2; -9; -9; 11; -5]
  Generated: [-6; 8; 0; 89; 8; -2; 6]
```

Putting them together

```
let property (xs : int list) : bool =
  take 0 xs = []

;; run_property_test
     "take 0 xs = []"
     (gen_list gen_int)
     property
```

```
PBT: take 0 xs = []

random seed: 11378409
generated error fail pass / total     time test name
[✓]  100    0    0  100 /  100     0.0s take 0 xs = []
================================================================
success (ran 1 tests)

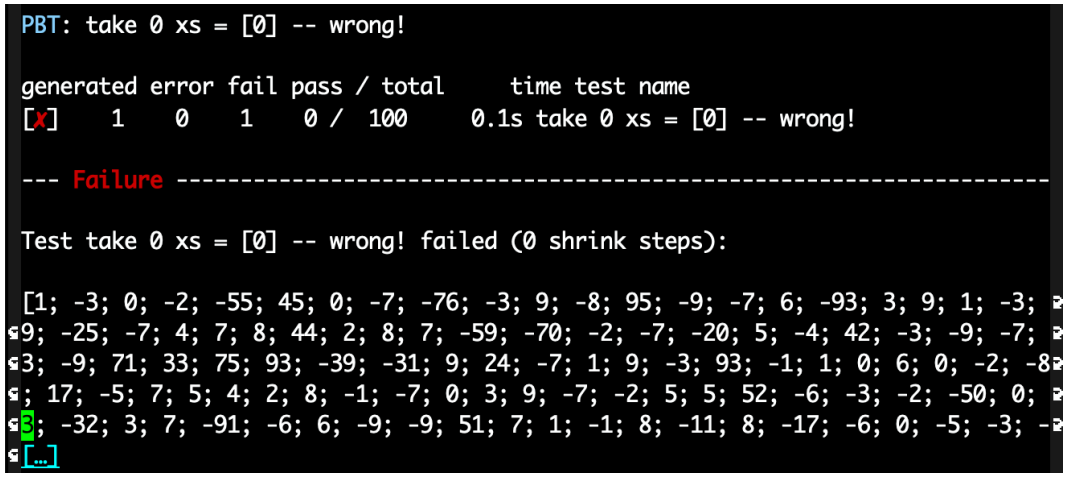
  --- take 0 xs = []: PASSED ---
```

A bad property

```
let wrong_property (xs : int list) : bool =
  take 0 xs = [0]

;; run_property_test
     "take 0 xs = [0] -- wrong!"
     (gen_list_no_shrinker gen_int_no_shrinker)
     wrong_property
```

An unreadable failing case

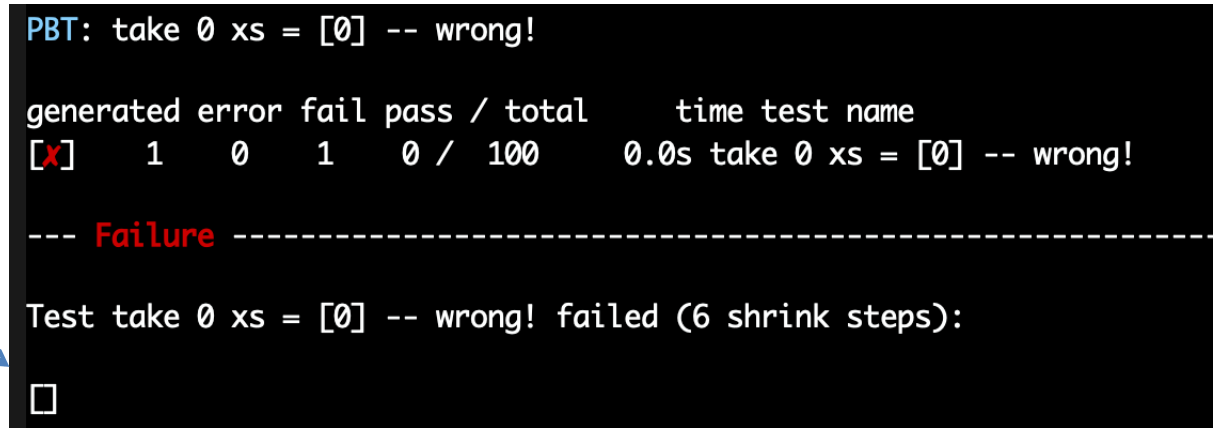


Shrinking

```
let wrong_property (xs : int list) : bool =
  take 0 xs = [0]

;; run_property_test
     "take 0 xs = [0] -- wrong!"
     (gen_list gen_int)
     wrong_property
```

A shrunk failing case



A more interesting test

```
let rec drop (n : int) (xs : int list) : int list =
  if n <= 0 then xs
  else begin match xs with
              | [] -> []
              | _ :: xs' -> drop (n - 1) xs'
       end

let property ((xs, n) : int list * int) : bool =
  take n xs @ drop n xs = xs

;; run_property_test
     "take n @ drop n"
     (gen_pair (gen_list gen_int) gen_int) property
```

Property-Based Testing

PBT = Property + Generator + Shrinking

Is PBT good for everything?

- No
- But it's extremely useful in some common situations…
  - Validity properties
    - Inserting into a BST yields a BST
  - Postcondition properties
    - ”take 0 xs is always empty”
  - Round-trip properties
    - Take/drop
    - Unzip / zip
    - Printing / parsing
  - Combinations of operations
    - Inserting an element into a BST and then deleting it does not change the set of elements